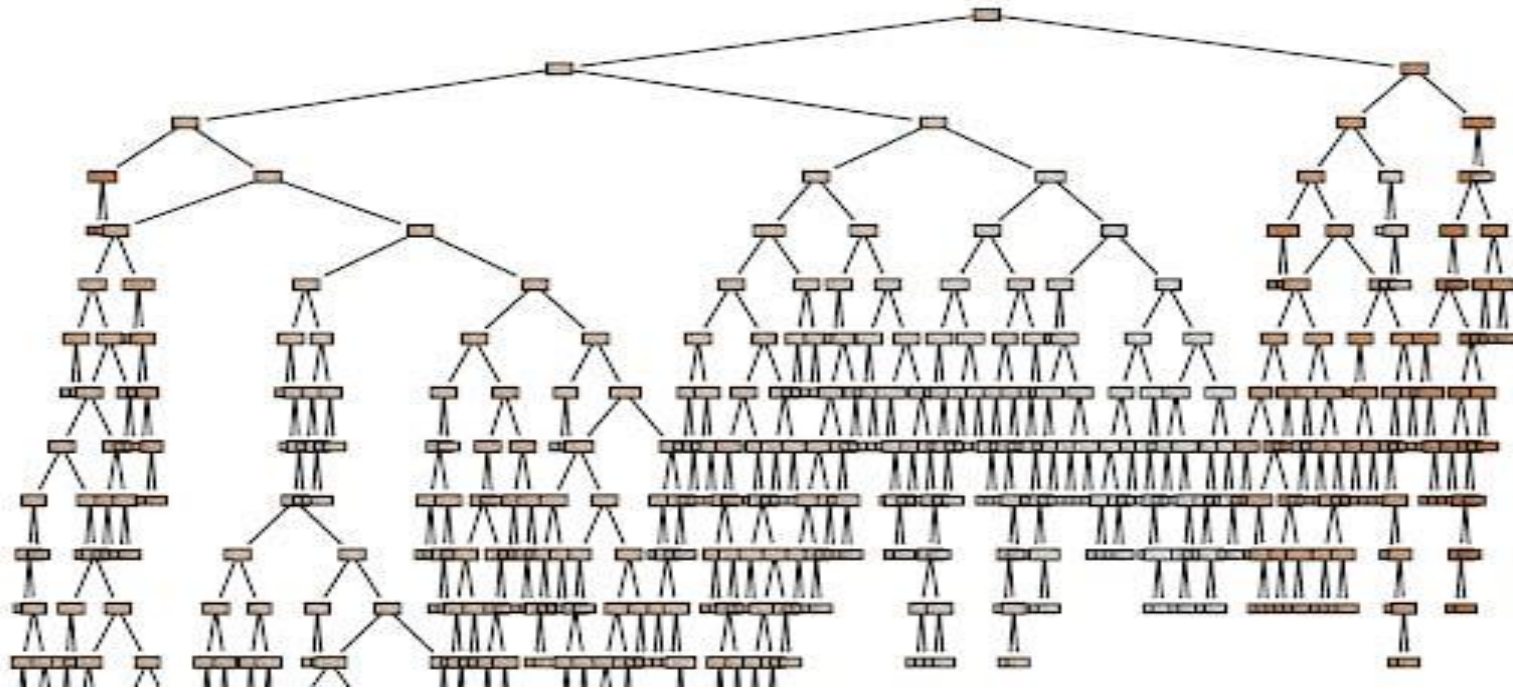


Otros modelos basados en
árboles de decisión:
Boosting y bagging

Cuidado con los árboles de decisión

Los árboles de decisión pueden caer muy fácil en overfitting si memorizan todos los datos de entrenamiento.



Potenciando los árboles de decisión: Boosting y Bagging

Los árboles de decisión, a pesar de ser modelos decentes, no son tan buenos generalizándose a varios problemas y a resultados fuera del conjunto de entrenamiento.

Para resolver esto, se utilizan técnicas que **combinan** varios árboles de decisión (en vez de uno solo), lo cual permitirá tener modelos mucho más precisos y aptos para cualquier escenario. Estos métodos que combinan varios modelos en uno solo se llaman **ensambles**.

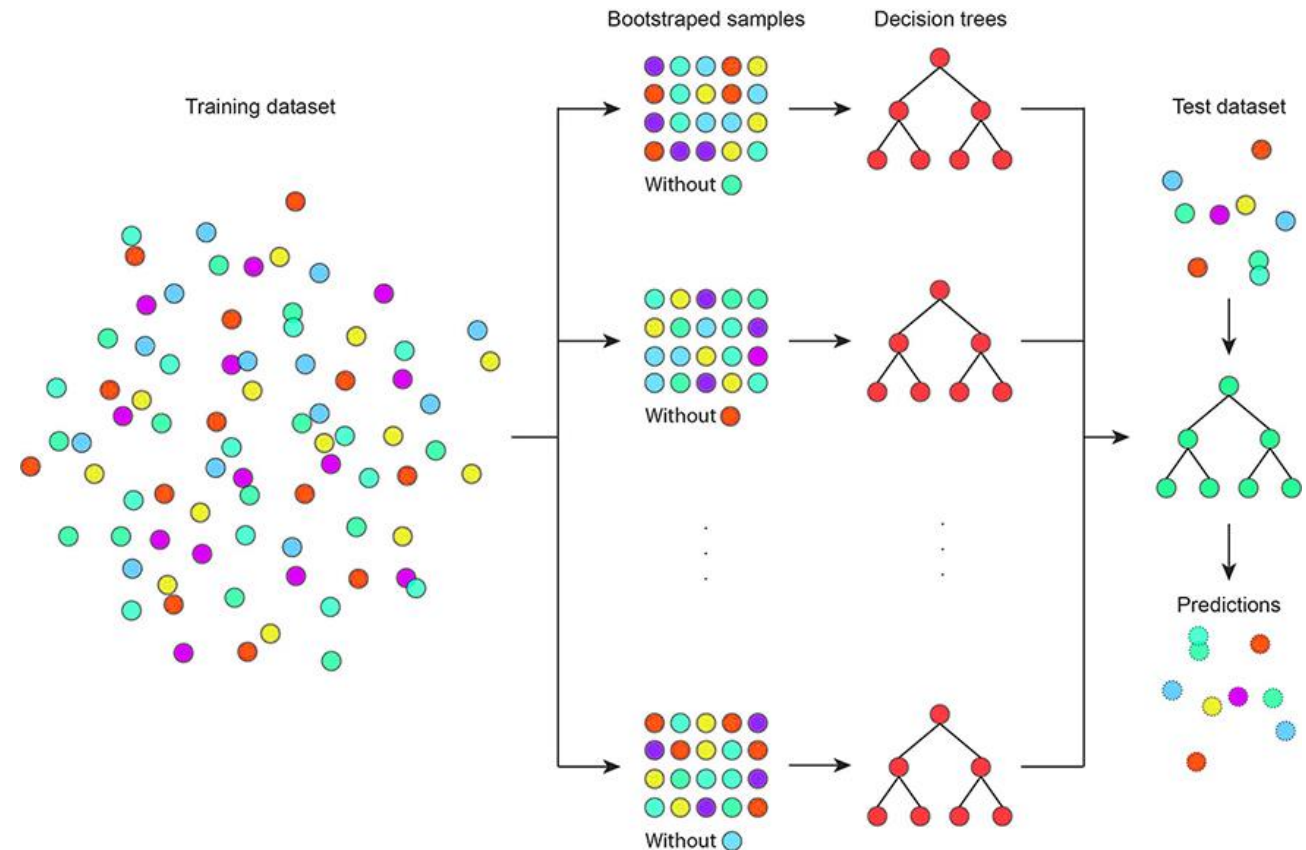
En particular usaremos dos tipos de modelos de ensamble que se conocen como **bagging y boosting**.

Bagging (Bootstrap aggregating).

Construir un “bosque” de árboles, en donde cada árbol del bosque trabaja de forma **independiente** y luego combinan sus salidas para generar la predicción.

Se llama bagging porque en cada árbol de forma aleatoria se usa una **submuestra con repetición (Bootstrap) de los datos** y al final se agregan.

Aptos para computación en **paralelo**.



Random forest

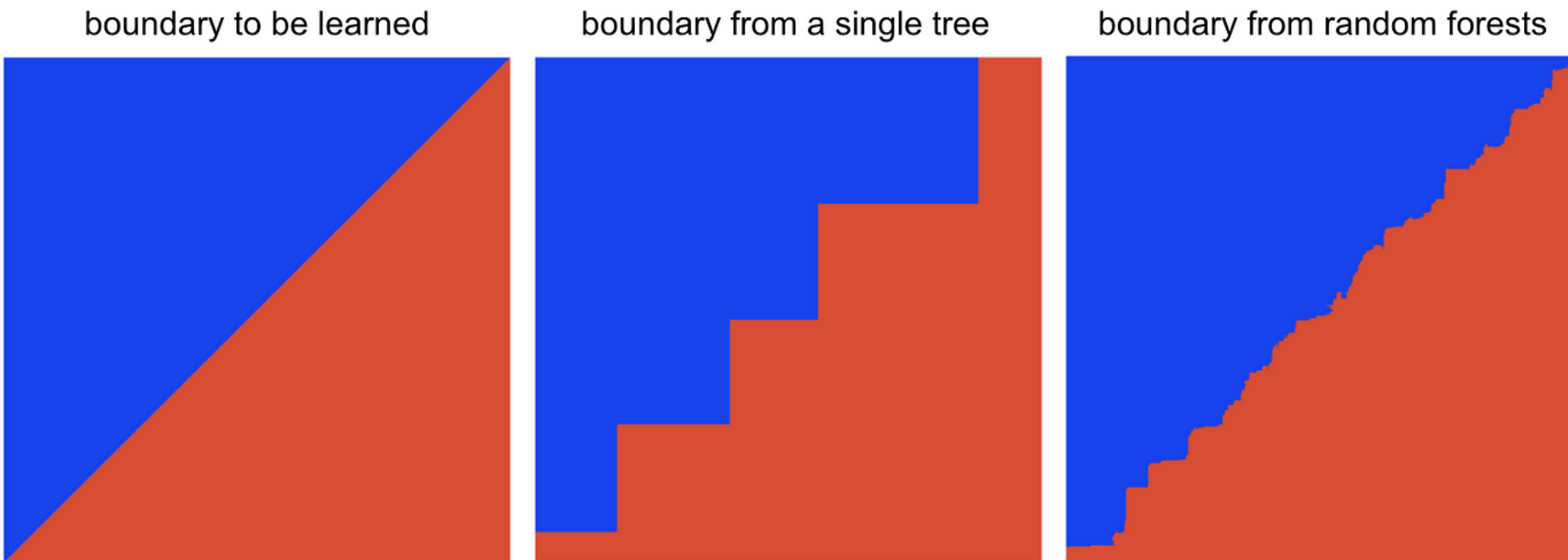
En este curso, usaremos un algoritmo llamado Random Forest, donde cada árbol del bosque usa un subconjunto de los registros y de las características para entrenarse (**bagging**).

Además, el random forest, al momento de hacer cada una de las divisiones del árbol, aleatoriamente solo considera un subconjunto de las variables para entre ellas elegir la mejor variable para hacer la división, para así tener aún más diversidad en el bosque.

Random forest: puliendo la predicción

El bosque agregado de predicciones ayuda a pulir mucho más los límites para mejorar la predicción.

Animación interesante para ver: <https://towardsdatascience.com/why-random-forests-outperform-decision-trees-1b0f175a0b5>



Random forest: tunearlo

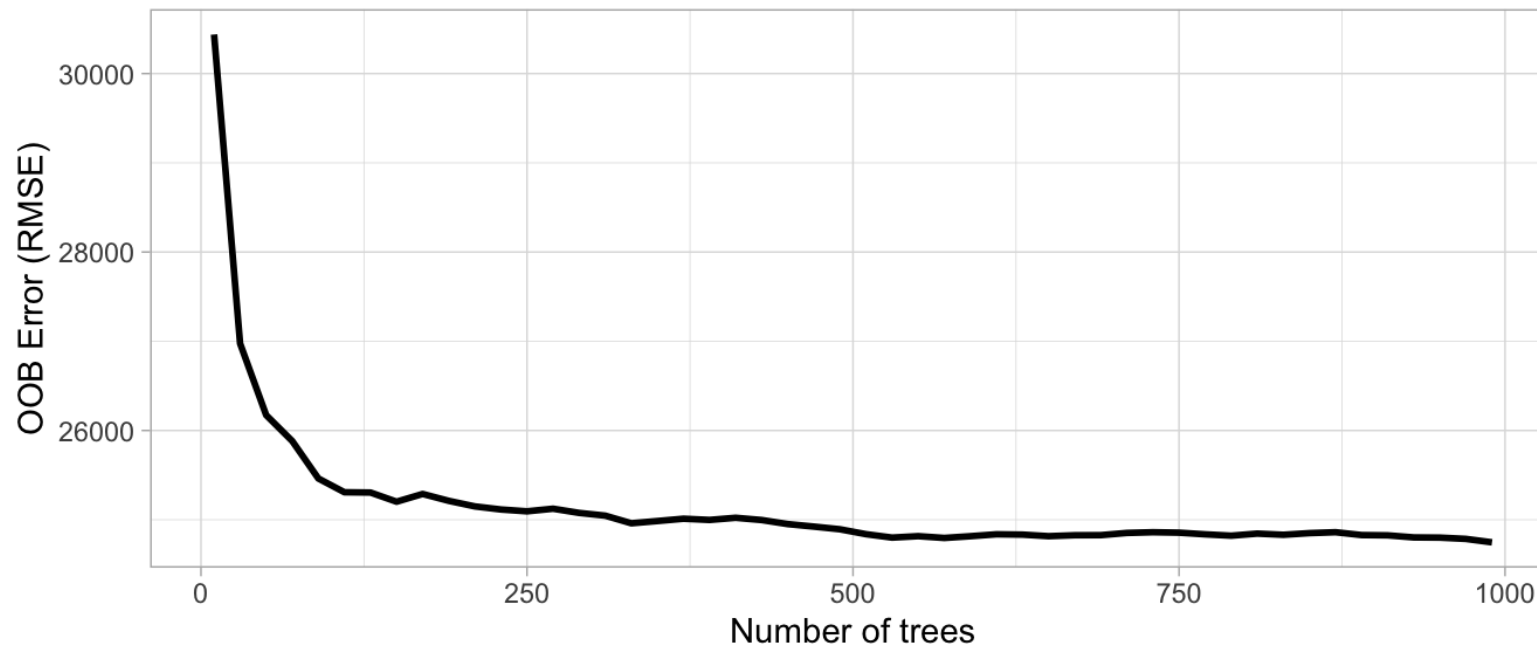
Usualmente, se espera que cada uno de los árboles en random forest tenga una profundidad relativamente **alta** para que cada uno de ellos esté minimizando el **bias** (el error).

Luego, al combinar las predicciones de todos y buscar un consenso, se va a reducir mucho la variabilidad en estas (porque se toma la decisión que tome la mayoría), reduciendo así también la **varianza**.

Así se logra tener bias y varianza bajos, lo cual es ideal.

Random forest: tunearlo

En general, se recomienda entonces usar una profundidad **alta** en los árboles y un número **adecuado** de árboles en el bosque (si se usan demasiados árboles, el tiempo de cómputo y las mejoras en error dejarán de ser significativas).



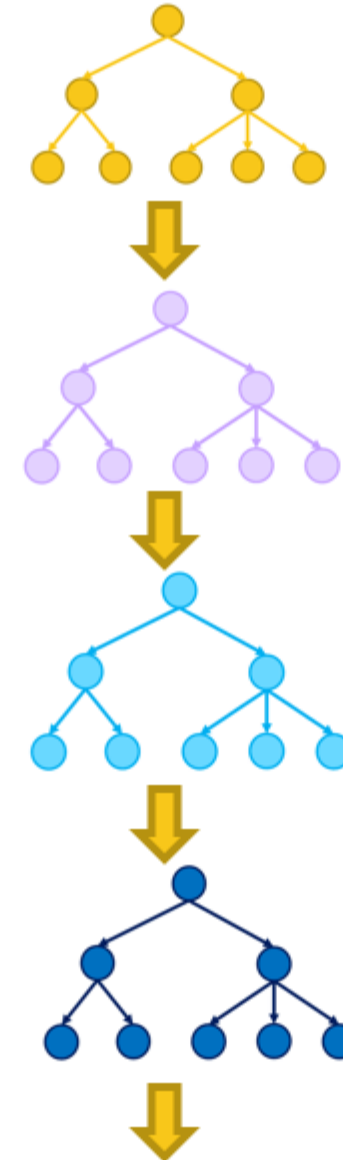
Boosting

En este caso, al contrario de lo que se hacía en bagging, se crea un conjunto **en serie** de árboles sucesivos (modelos).

Cada uno de los árboles **sucesivos** debe operar **a partir del trabajo hecho por el anterior árbol**.

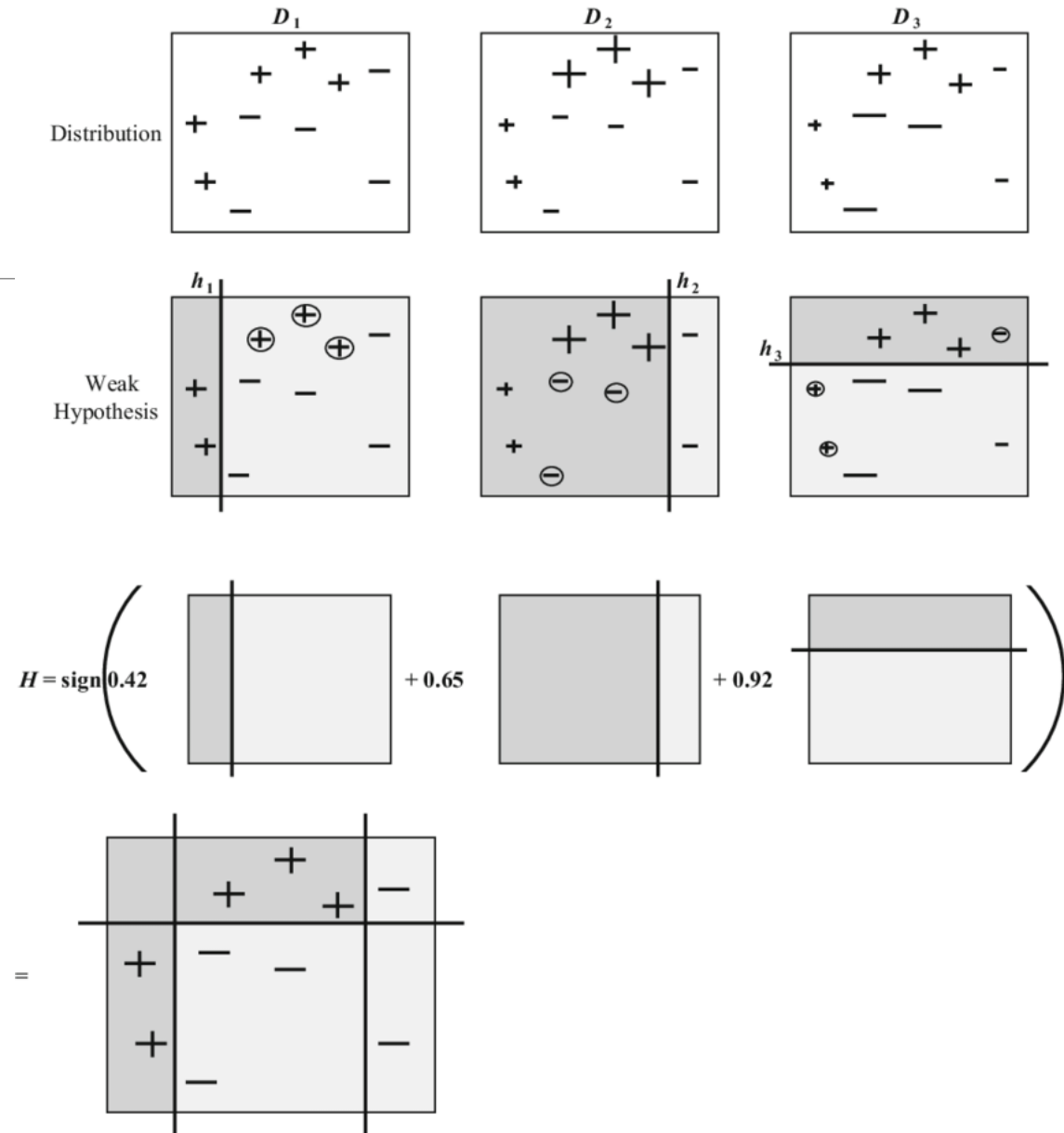
Así se espera ir mejorando progresivamente la predicción (se va puliendo poco a poco a medida que pasa por los árboles).

Gradient Boosted Trees



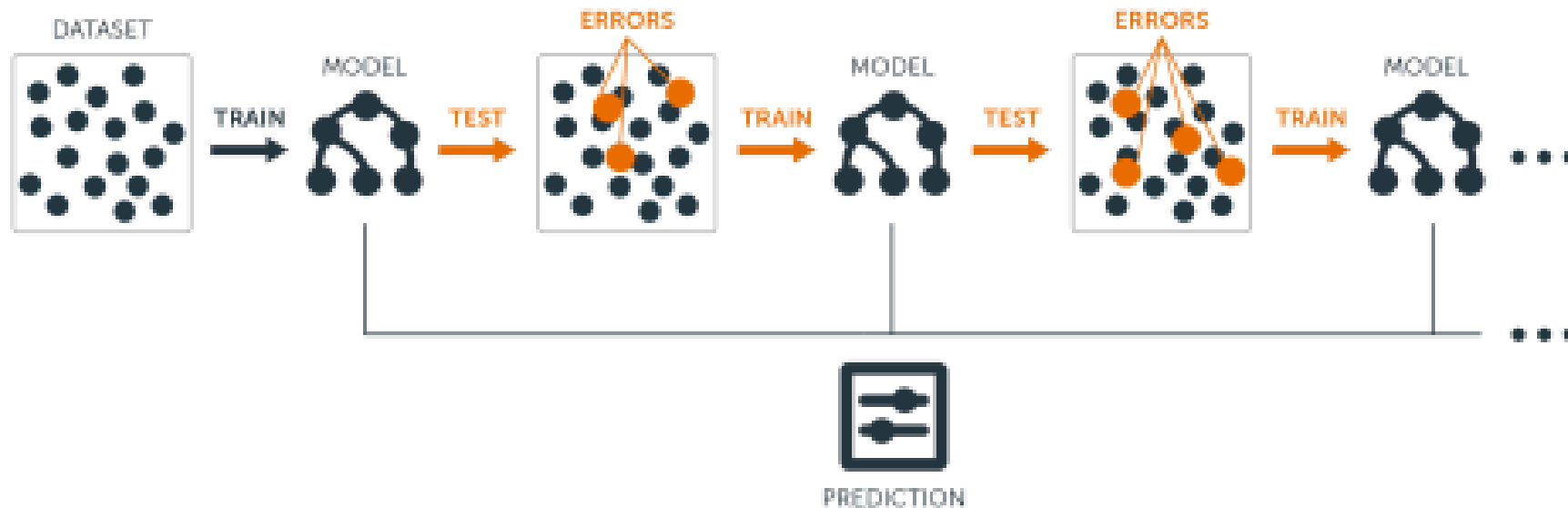
AdaBoost

El caso más básico de boosting es adaboost, donde se ponderan las observaciones en cada etapa según qué tan grande es su error de predicción.



Gradient Boosting

En este curso, el algoritmo de boosting que usaremos es llamado **Gradient Boosting**, donde cada árbol de la sucesión utiliza los residuos de los árboles anteriores (de ahí su analogía a los “gradientes” del modelo).



Gradient Boosting: tunearlo

Uualmente, se espera que cada uno de los árboles en gradient boosting tengan un grado de profundidad relativamente **bajo**, para que estén minimizando la varianza (no se compliquen demasiado).

Luego, los niveles sucesivos en serie tendrán cada uno árboles simples (weak learners), pero al pasar por todos vamos de a poco mejorando cada vez más la predicción final del modelo (reduce bias), logrando así tener bias y varianza bajos.

En general, se recomienda usar una profundidad **baja** en cada uno de los árboles individuales y usar un número **adecuado** de árboles en el ensamble (al igual que en random forest, tampoco se debe usar un número demasiado alto de árboles).

Versiones aún más avanzadas

Existen versiones aún más avanzadas de gradient boosting, llamadas por ejemplo XGboost (extreme gradient boosting) y LGBM (light gradient boosting machine).

Estas versiones avanzadas suelen mejorar principalmente la velocidad de cómputo y el manejo de memoria de los modelos, además de incluir algunas regularizaciones para evitar el overfitting. Este aumento de la velocidad hace que se puedan evaluar muchas más combinaciones de hiperámetros.

Versiones aún más avanzadas

Estos modelos avanzados son los caballos de batalla de muchos expertos en Kaggle y suelen ganar muchas competencias de machine learning.

Competition	Type	Winning ML Library/Algorithm
Liberty Mutual	Regression	XGBoost
Caterpillar Tubes	Regression	Keras + XGBoost + Reg. Forest
Diabetic Retinopathy	Image	SparseConvNet + RF
Avito	CTR	XGBoost
Taxi Trajectory 2	Geostats	Classic neural net
Grasp and Lift	EEG	Keras + XGBoost + other CNN
Otto Group	Classification	Stacked ensemble of 35 models
Facebook IV	Classification	sklearn GBM

Un pequeño vistazo a otros
modelos de machine
learning supervisado

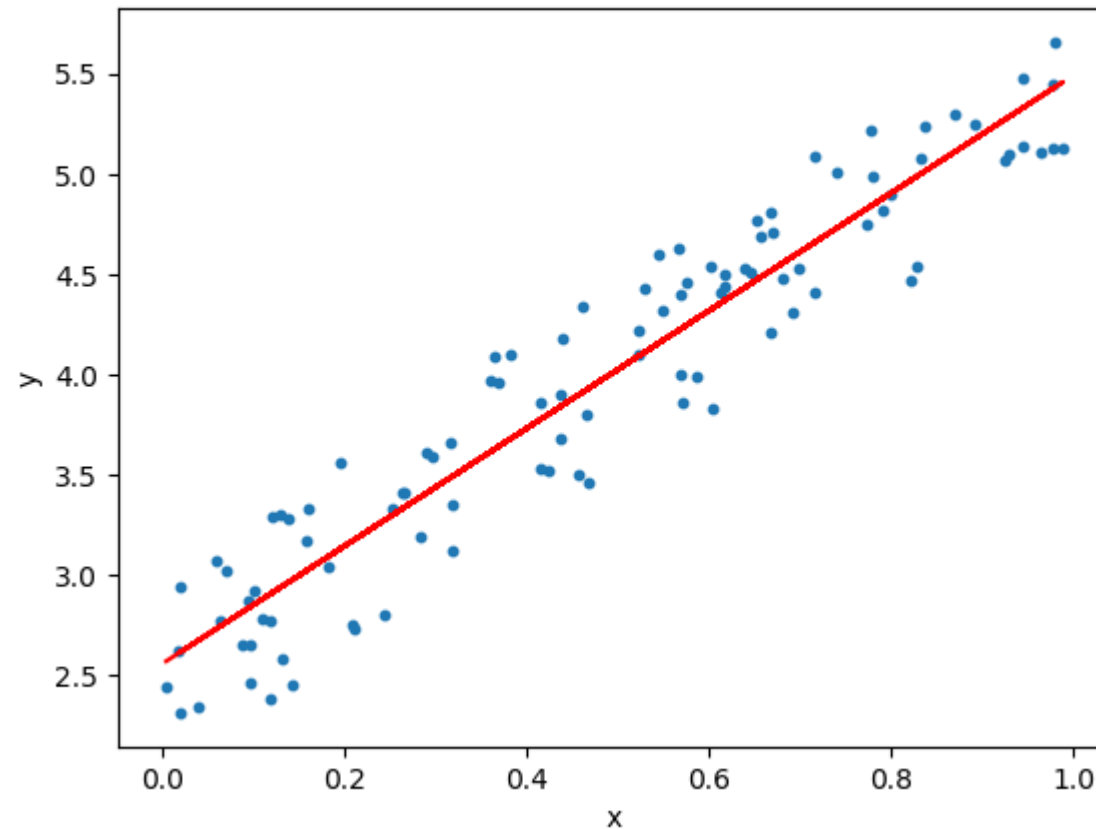
Otros tipos de modelos

Para este curso, nos enfocaremos en estos tres tipos de modelos para la parte de modelado (árboles de decisión, gradient boosting y random forest).

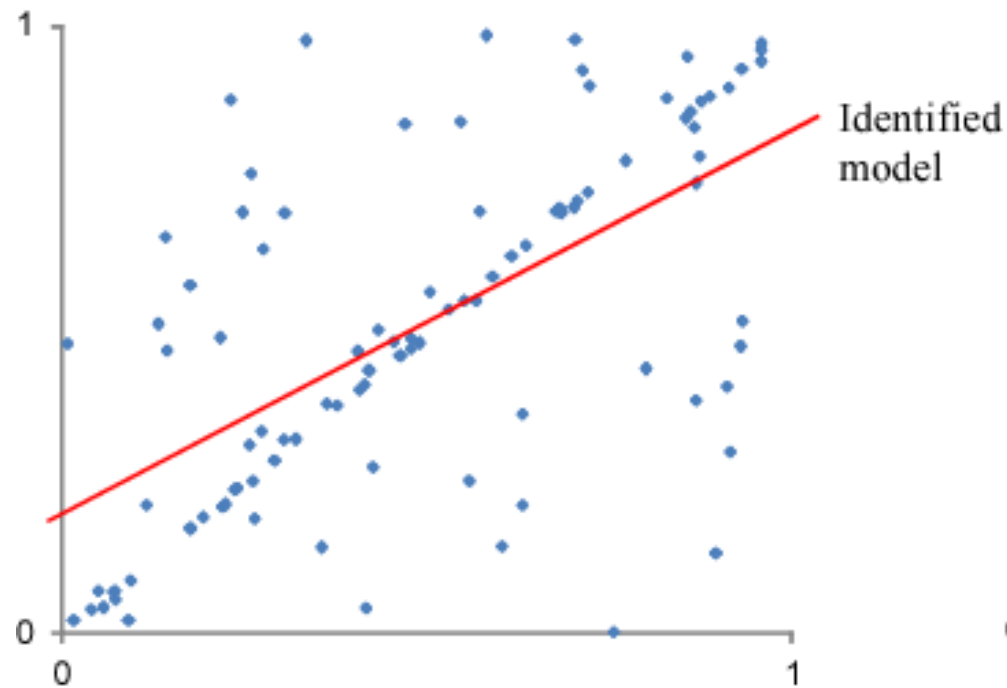
Sin embargo, existen muchísimos otros tipos de modelos que también sirven para problemáticas similares.

En un problema de analítica real será necesario probar también muchos otros tipos de modelos en busca de encontrar el mejor.

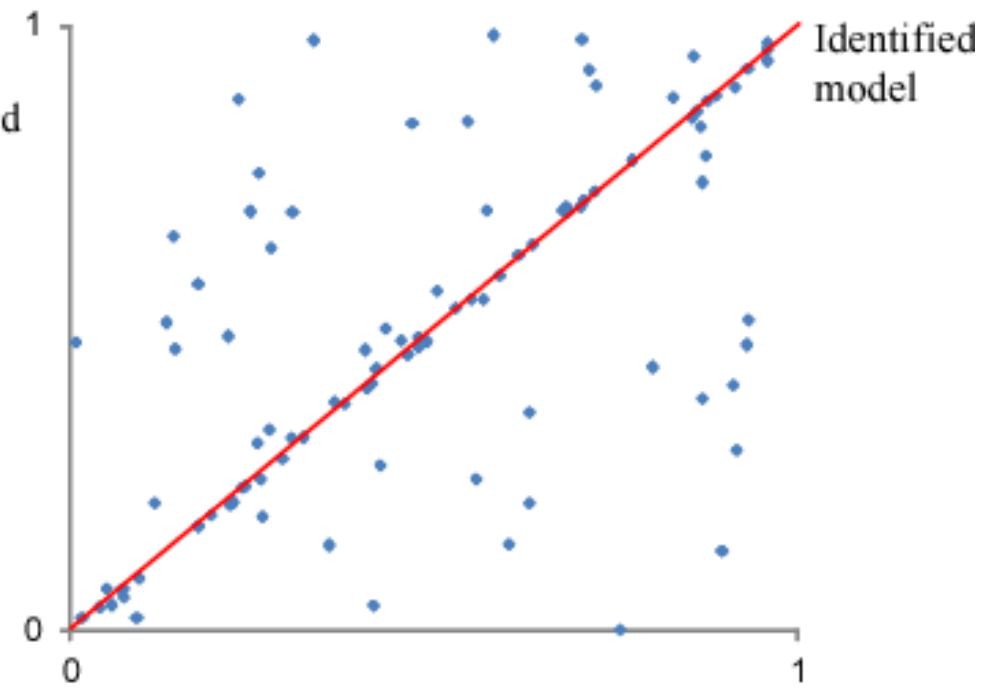
Regresión lineal



Regresión lineal robusta

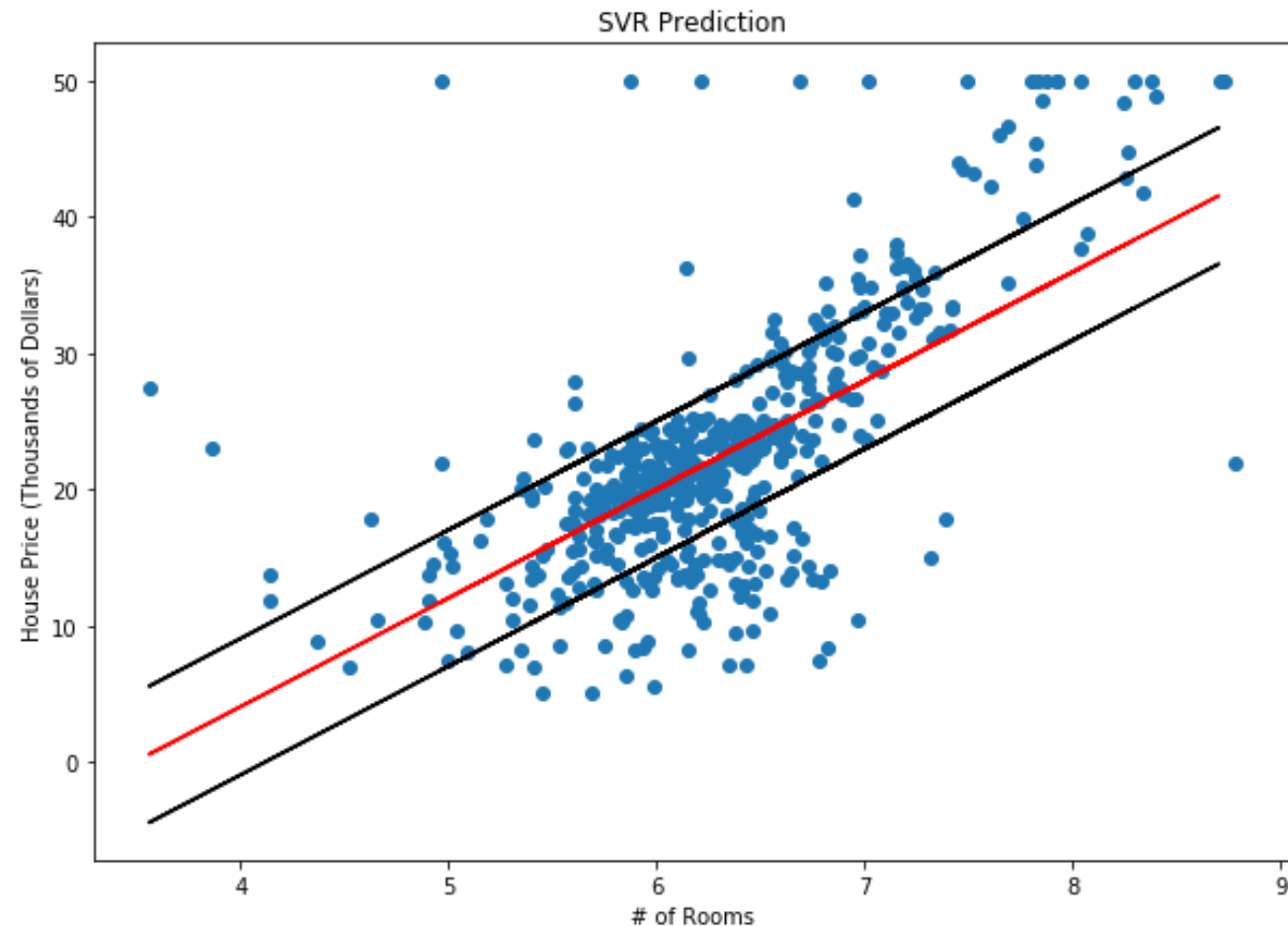


(a) Result by linear regression

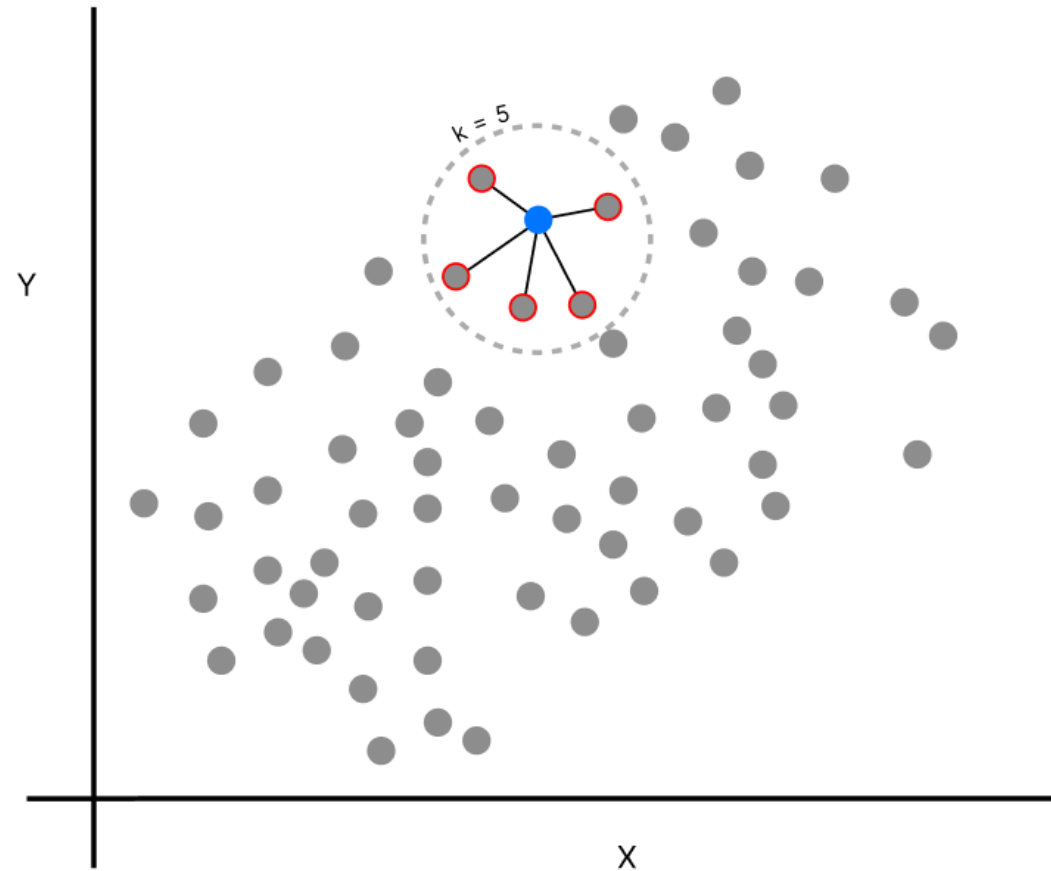


(b) Result by proposed RANSAC

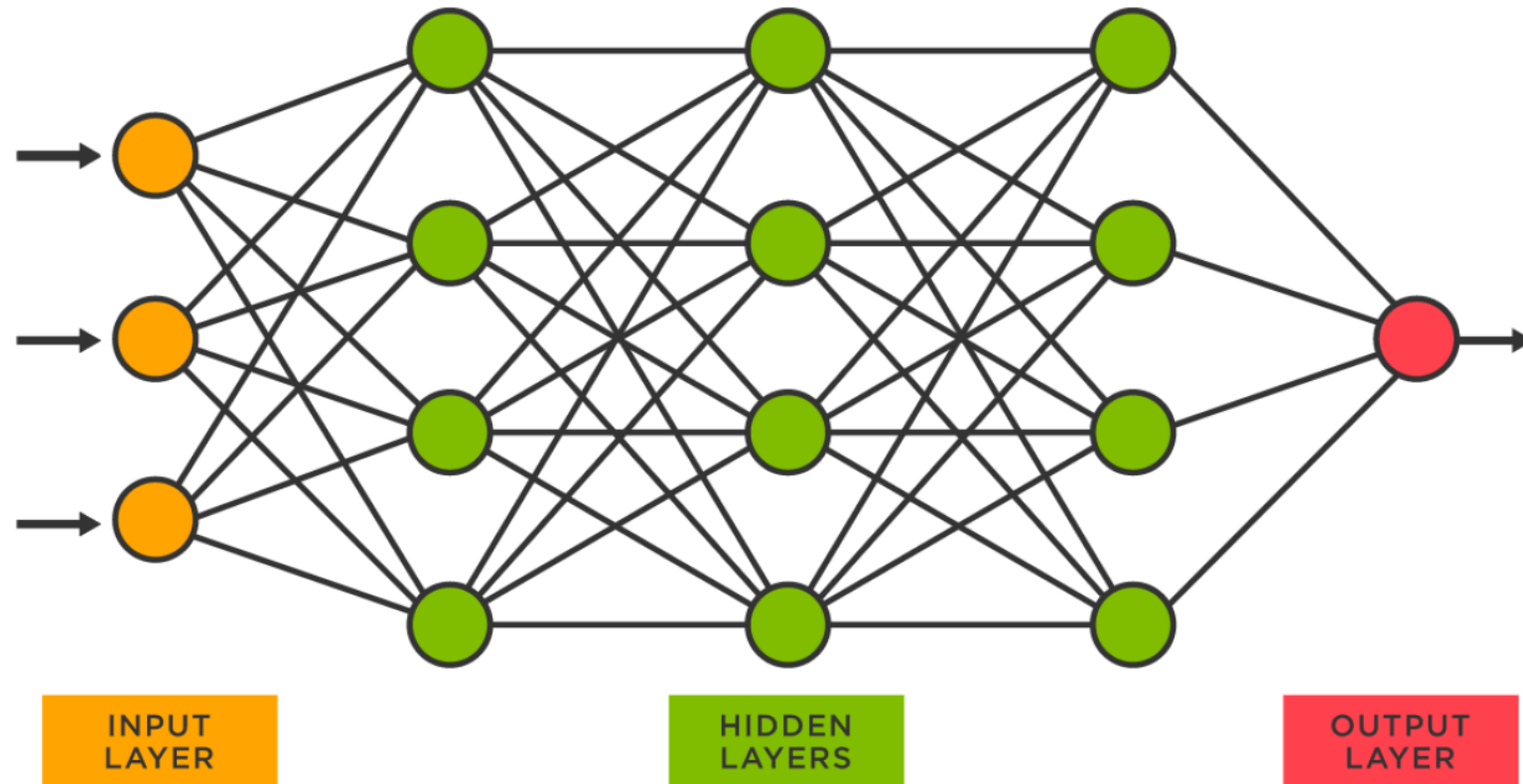
Support vector machine



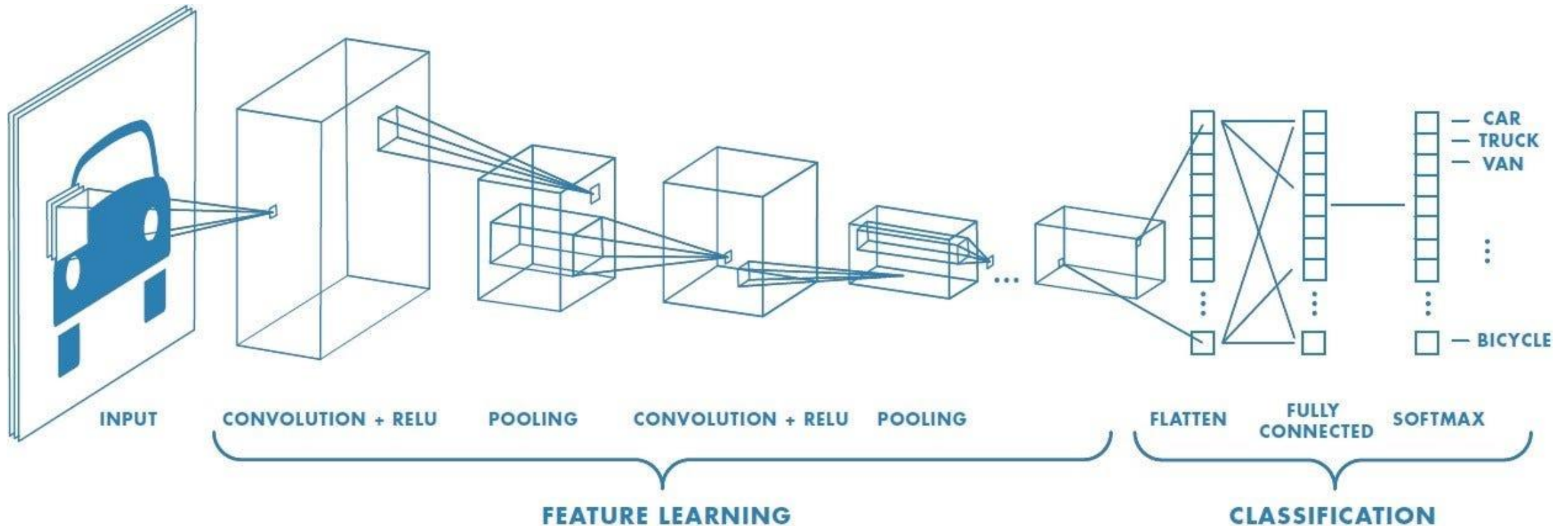
K-nearest neighbors



Redes neuronales



Deep learning



Y muchos más...

